

Python 프로그래밍

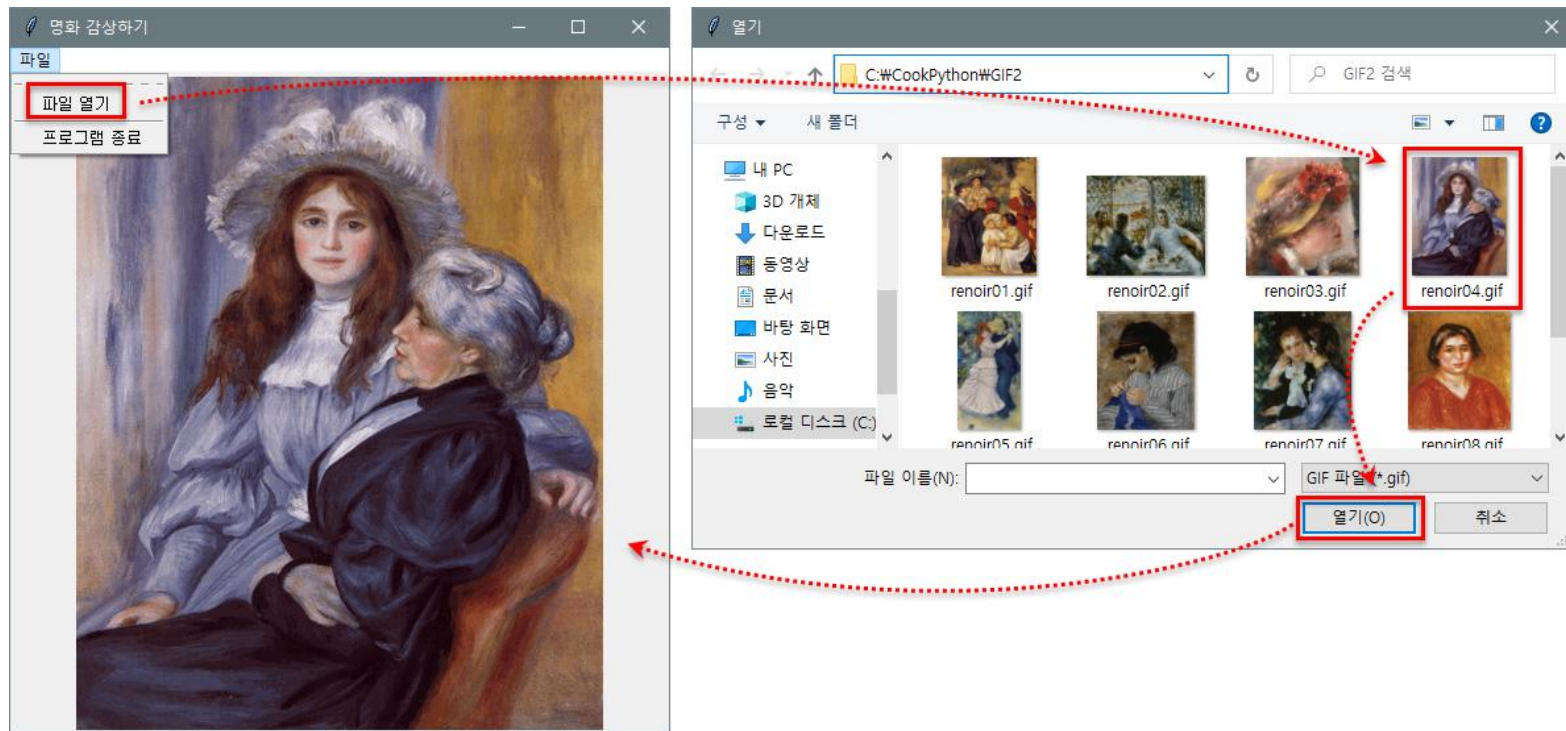
2026년 여름학기

- 기본 위젯 사용법을 이해한다.
- 위젯의 배치와 크기 조절 등의 활용방법을 이해한다.
- 키보드, 마우스 이벤트 처리 방법을 이해한다.
- 메뉴와 대화상자 구현 방법을 이해한다.

■ [프로그램 1] 사진 앨범



■ [프로그램 2] 영화 감상(파일 열기)

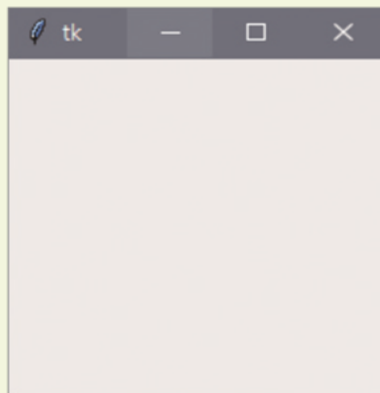


■ 기본 윈도우창의 구성

- 위젯(Widget) : 윈도우창에 나올 수 있는 문자, 버튼, 체크박스, 라디오버튼 등을 의미

Code10-01.py

```
1 from tkinter import *
2
3 window = Tk()
4
5 ## 이 부분에서 화면을 구성하고 처리 ##
6
7 window.mainloop()
```



Tip

- tkinter 는 TK Interface의 약어. Tk는 Tcl/Tk라는 전통적인 GUI 인터페이스 윈도우, 리눅스, 맥 등에서 모두 동일한 코드로 사용 가능

■ 윈도우창 조절

Code10-02.py

```
1 from tkinter import *
```

```
2
```

```
3 window = Tk()
```

```
4 window.title("윈도창 연습")
```

```
5 window.geometry("400x100")
```

```
6 window.resizable(width = FALSE, height = FALSE)
```

```
7
```

```
8 window.mainloop()
```

4행 : 윈도우창에 제목 표시

5행 : 윈도우창의 초기 크기를 400x100으로 지정

6행 : 가로와 세로의 크기가 변경되지 않도록 지정



크기 변경 안 됨

■ 레이블

- 레이블(Label) : 문자를 표현할 수 있는 위젯

Font : 글꼴 과 크기 지정

Fg : foreground의 약어로 글자색을 지정

Bg : background의 약어로 배경색을 지정

width와 height : 위젯의 폭과 높이를 지정

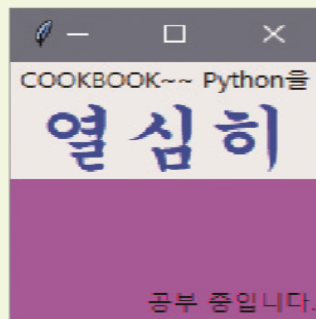
Anchor : 위젯이 어느 위치에 자리 잡을지 결정

Code10-03.py

```
1 from tkinter import *
2 window = Tk()
3
4 label1 = Label(window, text = "COOKBOOK~~ Python을")
5 label2 = Label(window, text = "열심히", font = ("궁서체", 30), fg = "blue")
6 label3 = Label(window, text = "공부 중입니다.", bg = "magenta", width = 20, height = 5,
7               anchor = SE)
8
9 label1.pack()
10 label2.pack()
11 label3.pack()
12 window.mainloop()
```

4행 : Label() 함수가 레이블을 만들어 줌

8행 : pack() 함수로 해당 레이블을 화면에 표시



Section 02 기본 위젯 활용

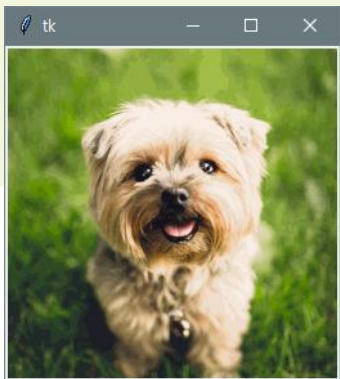
■ 레이블에 글자 대신 이미지 넣기

- PhotoImage()는 GIF 파일만 지원, JPEG나 BMP 등은 지원하지 않음

Code10-04.py

```
1 from tkinter import *
2 window = Tk()
3
4 photo = PhotoImage(file = "gif/dog.gif")
5 label1 = Label(window, image = photo)
6
7 label1.pack()
8
9 window.mainloop()
```

4행 : PhotoImage()에서 이미지 파일을 준비
5행 : Label()의 옵션 image에서 속성을 지정



■ 버튼

- 버튼(Button) : 마우스로 클릭하면 놀리는 효과와 함께 지정한 작업 실행
- 예: 버튼을 누르면 파이썬 IDLE이 종료되는 코드

Code10-05.py

```
1 from tkinter import *
2 window = Tk()
3
4 button1 = Button(window, text = "파이썬 종료", fg = "red", command = quit)
5
6 button1.pack()
7
8 window.mainloop()
```

4행 : command 옵션에 quit 명령



Section 02 기본 위젯 활용

- 예: 이미지 버튼을 누르면 간단한 메시지창이 나오는 코드

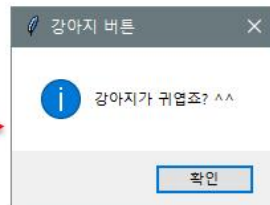
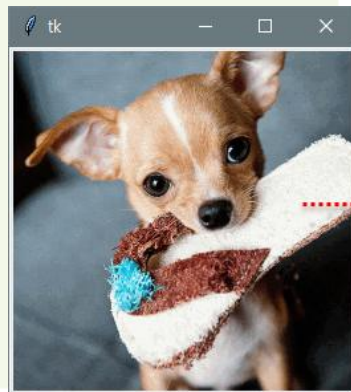
Code10-06.py

```
1 from tkinter import *
2 from tkinter import messagebox
3
4 ## 함수 선언 부분 ##
5 def myFunc() :
6     messagebox.showinfo("강아지 버튼", "강아지가 귀엽죠? ^^")
7
8 ## 메인 코드 부분 ##
9 window = Tk()
10
11 photo = PhotoImage(file = "gif/dog2.gif")
12 button1 = Button(window, image = photo, command = myFunc)
13
14 button1.pack()
15
16 window.mainloop()
```

2행 : messagebox 모듈 사용 импорт

5~6행 : myFunc() 함수 정의, 이 함수는messagebox.showinfo(제목, 내용) 형식으로 화면에 메시지 상자 출력

11~12행 : 이미지 준비하고 버튼에 글자 대신 이미지 표현

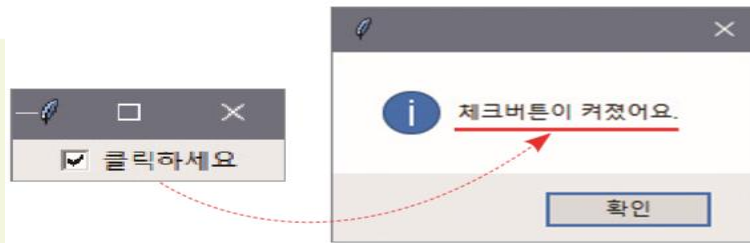


■ 체크버튼

- 체크버튼(Checkbutton) : 켜고 끄는 데 사용하는 위젯
- 예 : 체크버튼을 켜거나 끄면 메시지창이 열리게 하는 코드

Code10-07.py

```
1 from tkinter import *
2 from tkinter import messagebox
3 window = Tk()
4
5 ## 함수 선언 부분 ##
6 def myFunc() :
7     if chk.get() == 0 :
8         messagebox.showinfo("", "체크버튼이 꺼졌어요.")
9     else :
10        messagebox.showinfo("", "체크버튼이 켜졌어요.")
11
12 ## 메인 코드 부분 ##
13 chk = IntVar()
14 cb1 = Checkbutton(window, text = "클릭하세요", variable = chk, command = myFunc)
15
16 cb1.pack()
17
18 window.mainloop()
```



6~10행 : myFunc() 함수는 chk.get() 함수,
체크버튼에 설정된 값을 가져와서
메시지 출력

13행 : chk 변수를 준비, IntVar() 함수 사용

14행 : Checkbutton()의 옵션 중 variable에 chk 변수 사용

■ 라디오버튼

- 라디오버튼(Radiobutton) : 여러 개 중에서 하나를 선택하는 데 사용하는 위젯

Code10-08.py

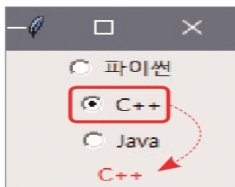
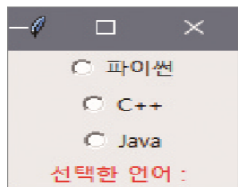
```
1  from tkinter import *
2  window = Tk()
3
4  ## 함수 선언 부분 ##
5  def myFunc() :
6      if var.get() == 1 :
7          label1.configure(text = "파이썬")
8      elif var.get() == 2 :
9          label1.configure(text = "C++")
10     else :
11         label1.configure(text = "Java")
12
13  ## 메인 코드 부분 ##
14  var = IntVar()
15  rb1 = Radiobutton(window, text = "파이썬", variable = var, value = 1, command = myFunc)
16  rb2 = Radiobutton(window, text = "C++", variable = var, value = 2, command = myFunc)
17  rb3 = Radiobutton(window, text = "Java", variable = var, value = 3, command = myFunc)
```

5~11행 : myFunc() 함수는 var 변수값으로 맨 아래쪽
레이블의 텍스트를 변경

14행 : 정수형 변수 var를 준비
15~17행 : 라디오버튼을 3개 준비

Section 02 기본 위젯 활용

```
18
19 label1 = Label(window, text = "선택한 언어 : ", fg = "red")
20
21 rb1.pack()
22 rb2.pack()
23 rb3.pack()
24 label1.pack()
25
26 window.mainloop()
```



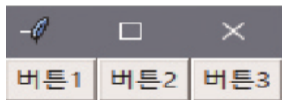
■ 수평 정렬

- 수평으로 정렬하는 방법 : pack() 함수의 옵션 중 side=LEFT, RIGHT

Code10-09.py

```
1 from tkinter import *
2 window = Tk()
3
4 button1 = Button(window, text = "버튼1")
5 button2 = Button(window, text = "버튼2")
6 button3 = Button(window, text = "버튼3")
7
8 button1.pack(side = LEFT)
9 button2.pack(side = LEFT)
10 button3.pack(side = LEFT)
11
12 window.mainloop()
```

8~10행 : side=LEFT 옵션은 왼쪽부터 채우라는 의미



■ 리스트와 for 문 활용

Code10-10.py

```
1 from tkinter import *
2 window = Tk()
3
4 btnList = [None] * 3
5
6 for i in range(0, 3) :
7     btnList[i] = Button(window, text = "버튼" + str(i + 1))
8
9 for btn in btnList :
10     btn.pack( side = RIGHT )
11
12 window.mainloop()
```

4행 : 비어 있는 리스트를 3개 준비

6~7행 : 3회 반복하면서 버튼 생성

9~10행 : 준비한 버튼 리스트를 화면에 출력

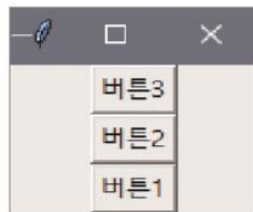


■ 수직 정렬

- pack() 함수의 옵션 중 수직으로 정렬하는 방법 : side=TOP, BOTTOM
 - Code10-10.py 의 10행을 다음과 같이 변경하고 실행



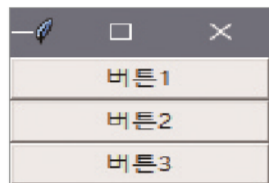
```
btn.pack(side = TOP)
```



```
btn.pack(side = BOTTOM)
```

■ 폭 조정

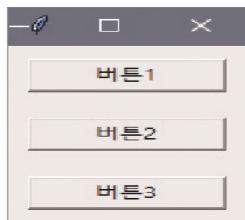
- pack() 함수의 옵션 중에서 윈도우 폭에 맞추는 방법 : fill=X이다.
 - Code10-10.py의 10행을 다음과 같이 변경하고 실행



```
btn.pack(side = TOP, fill = X)
```


■ 위젯 사이의 여백 조절

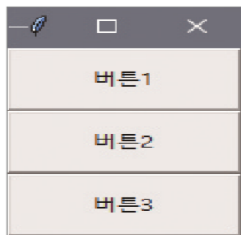
- pack() 함수의 옵션 중에서 위젯 사이에 여백 주는 방법 : padx=픽셀값 또는 pady=픽셀값
 - Code10-10.py의 10행을 다음과 같이 변경하고 실행



```
btn.pack(side = TOP, fill = X, padx = 10, pady = 10)
```

■ 위젯 내부의 여백 조절

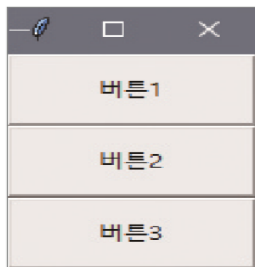
- pack() 함수의 옵션 중에서 위젯 내부에 여백 주는 방법 : ipadx=픽셀값 또는 ipady=픽셀값
 - Code10-10.py의 10행을 다음과 같이 변경하고 실행



```
btn.pack(side = TOP, fill = X, ipadx = 10, ipady = 10)
```

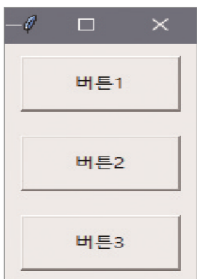
■ 위젯 내부의 여백 조절

- pack() 함수의 옵션 중에서 위젯 내부에 여백 주는 방법 : ipadx=픽셀값 또는 ipady=픽셀값
 - Code10-10.py의 10행을 다음과 같이 변경하고 실행



```
btn.pack(side = TOP, fill = X, ipadx = 10, ipady = 10)
```

■ 위젯 내부와 외부에 모두 여백



```
btn.pack(side = TOP, fill = X, ipadx = 10, ipady = 10, padx = 10, pady = 10)
```

■ 고정 위치에 배치

- 위젯을 고정 위치에 배치하려면 pack() 대신 place() 함수 사용
- 그림 9개를 2차원으로 배치하는 코드

Code10-11.py

```
1  from tkinter import *
2
3  ## 전역 변수 선언 부분 ##
4  btnList = [None] * 9
5  fnameList = ["froyo.gif", "gingerbread.gif", "honeycomb.gif", "icecream.gif",
6               "jellybean.gif", "kitkat.gif", "lollipop.gif", "marshmallow.gif", "nougat.gif"]
7  photoList = [None] * 9
8  i, k = 0, 0
9  xPos, yPos = 0, 0
10 num = 0
11
12 ## 메인 코드 부분 ##
13 window = Tk()
14 window.geometry("210x210")
15
16 for i in range(0, 9) :
17     photoList[i] = PhotoImage(file = "gif/" + fnameList[i])
18     btnList[i] = Button(window, image = photoList[i])
```

4행 : 버튼을 저장할 9개짜리 리스트 준비

5행 : 이미지 파일명 9개를 리스트로 준비

6행 : PhotoImage()로 생성할 9개짜리 리스트

8행 : xPos, yPos는 그림을 출력할 X, Y 좌표로 사용

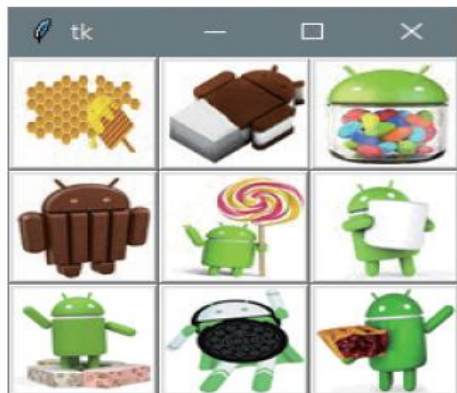
9행 : num 은 그림의 순차 번호로 사용

15~17행 : 9번 반복하면서 버튼을 생성

Section 03 위젯의 배치와 크기 조절

```
18
19 for i in range(0, 3) :
20     for k in range(0, 3) :
21         btnList[num].place(x = xPos, y = yPos)
22         num += 1
23         xPos += 70
24     xPos = 0
25     yPos += 70
26
27 window.mainloop()
```

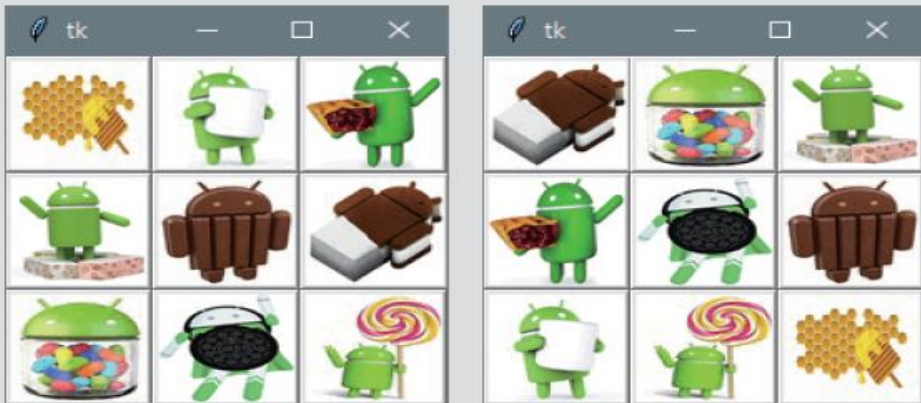
19~25행 : $3 \times 3 = 9$ 번 반복



SELF STUDY 10-2

Code10-11.py를 실행할 때마다 그림을 임의로 뒤섞어서 나타내게 하자.

힌트 random 모듈을 임포트하고 shuffle(리스트) 함수를 사용하면 리스트를 임의로 뒤섞어 준다.



■ [프로그램 1]의 완성

- <이전> 버튼이나 <다음> 버튼을 누르면 사진들을 표시하는 사진 앨범 프로그램

Code10-12.py

```
1  from tkinter import *
2  from time import *
3
4  ## 전역 변수 선언 부분 ##
5  fnameList = ["jeju1.gif", "jeju2.gif", "jeju3.gif", "jeju4.gif", "jeju5.gif",
               "jeju6.gif", "jeju7.gif", "jeju8.gif", "jeju9.gif"]
6  photoList = [None] * 9      5행 : fnameList 변수에 사진 9장의 파일명을 저장
7  num = 0                    6행 : photoList에는 PhotoImage() 함수로 생성할 변수 9개 준비
8                               7행 : num은 현재 사진의 번호
9  ## 함수 선언 부분 ##
10 def clickNext() :           10~17행 : <다음> 버튼을 누르면 실행되는 함수
11     global num              11행 : global num은 num 전역 변수를 함수 안에서 사용 의미
12     num += 1                12행 : 사진 번호를 하나 증가
13     if num > 8 :            13~14행 : 사진 번호가 최대 8, 8 넘으면 0번 사진 표시
```

Section 03 위젯의 배치와 크기 조절

```
14         num = 0
15         photo = PhotoImage(file = "gif/" + fnameList[num])
16         pLabel.configure(image = photo)
17         pLabel.image = photo
18
19     def clickPrev() :
20         global num
21         num -= 1
22         if num < 0 :
23             num = 8
24         photo = PhotoImage(file = "gif/" + fnameList[num])
25         pLabel.configure(image = photo)
26         pLabel.image = photo
27
28     ## 메인 코드 부분 ##
29     window = Tk()
30     window.geometry("700x500")
31     window.title("사진 앨범 보기")
```

15~17행 : 변경된 사진 번호에 해당하는
이미지 파일로 pLabel 변경

19~26행 : <이전> 버튼을 누르면 처리되는 함수

Section 03 위젯의 배치와 크기 조절

```
32
33 btnPrev = Button(window, text = "<< 이전", command = clickPrev)
34 btnNext = Button(window, text = "다음 >>", command = clickNext)
35
36 photo = PhotoImage(file = "gif/" + fnameList[0])
37 pLabel = Label(window, image = photo)
38
39 btnPrev.place(x = 250, y = 10)
40 btnNext.place(x = 400, y = 10)
41 pLabel.place(x = 15, y = 50)
42
43 window.mainloop()
```

33~34행 : 버튼 누르면 이 함수와 연결

36~37행 : 프로그램 실행 후 첫 번째
사진 표시

39~41행 : 버튼 및 이미지 위치
place(x=좌표, y=좌표)로 지정



■ 마우스 이벤트 기본 처리

마우스 작동	마우스 버튼	이벤트 코드
클릭할 때	모든 버튼 공통	〈Button〉
	왼쪽 버튼	〈Button-1〉
	가운데 버튼	〈Button-2〉
	오른쪽 버튼	〈Button-3〉
떼었을 때	모든 버튼 공통	〈ButtonRelease〉
	왼쪽 버튼	〈ButtonRelease-1〉
	가운데 버튼	〈ButtonRelease-2〉
	오른쪽 버튼	〈ButtonRelease-3〉

마우스 작동	마우스 버튼	이벤트 코드
더블클릭할 때	모든 버튼 공통	〈Double-Button〉
	왼쪽 버튼	〈Double-Button-1〉
	가운데 버튼	〈Double-Button-2〉
	오른쪽 버튼	〈Double-Button-3〉
드래그할 때	왼쪽 버튼	〈B1-Motion〉
	가운데 버튼	〈B2-Motion〉
	오른쪽 버튼	〈B3-Motion〉
마우스 커서가 위젯 위로 올라왔을 때		〈Enter〉
마우스 커서가 위젯에서 떠났을 때		〈Leave〉

Section 04 키보드와 마우스 이벤트 처리

- 마우스 이벤트가 처리 형식

```
def 이벤트처리함수(event) :  
    # 이 부분에 마우스 이벤트가 발생할 때 작동할 내용 작성  
  
    위젯.bind("마우스이벤트", 이벤트처리함수)
```

Section 04 키보드와 마우스 이벤트 처리

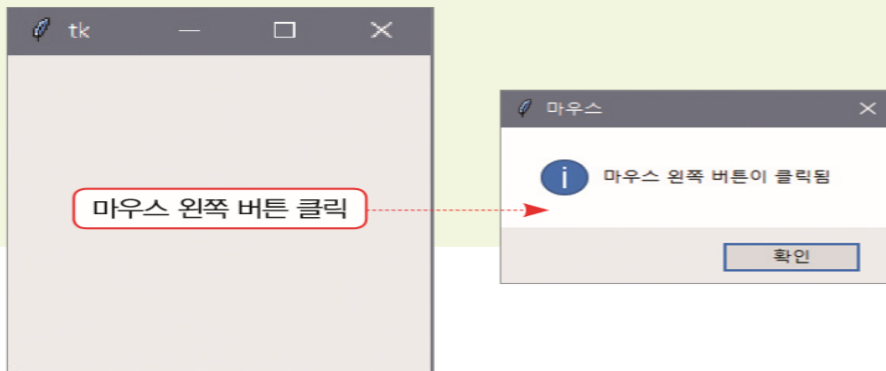
■ 마우스 왼쪽 버튼을 클릭했을 때 처리하는 방법

Code10-13.py

```
1 from tkinter import *
2 from tkinter import messagebox
3
4 ## 함수 선언 부분 ##
5 def clickLeft(event) :
6     messagebox.showinfo("마우스", "마우스 왼쪽 버튼이 클릭됨")
7
8 ## 메인 코드 부분 ##
9 window = Tk()
10
11 window.bind("<Button-1>", clickLeft)
12
13 window.mainloop()
```

5~6행 : 마우스 이벤트가 발생할 때 작동할 함수 정의

11행 : window.bind() 함수에는 마우스 왼쪽 버튼 클릭
할 때 발생하는 이벤트인 <Button-1> 설정하고
5행의 clickLeft 함수명을 지정

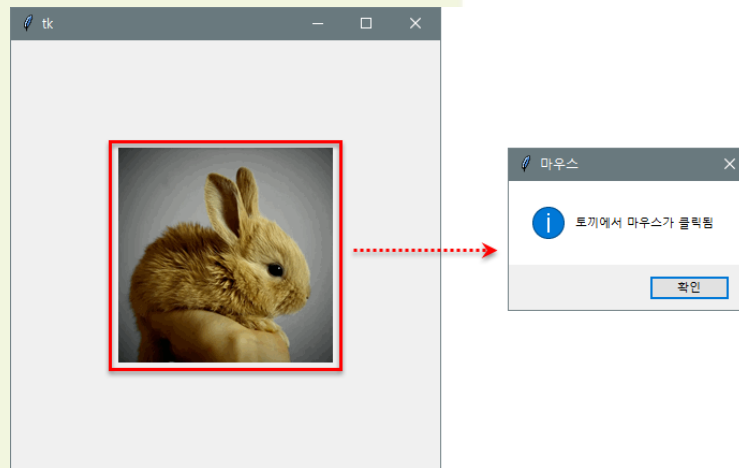


Section 04 키보드와 마우스 이벤트 처리

- 지정된 위젯을 클릭했을 때 다른 함수 호출

Code10-14.py

```
1  from tkinter import *
2  from tkinter import messagebox
3
4  ## 함수 선언 부분 ##
5  def clickImage(event) :
6      messagebox.showinfo("마우스", "토끼에서 마우스가 클릭됨")
7
8  ## 메인 코드 부분 ##
9  window = Tk()
10 window.geometry("400x400")
11
12 photo = PhotoImage(file = "gif/rabbit.gif")
13 label1 = Label(window, image = photo)
14
15 label1.bind("<Button>", clickImage)
16
17 label1.pack(expand = 1, anchor = CENTER)
18 window.mainloop()
```



이미지 클릭할 때만 이벤트 처리(15행에서 window.bind()가 아닌 label1.bind()를 사용했기 때문) 또 <Button> 이벤트를 사용했기 때문에 어떤 마우스 버튼 눌러도 메시지가 표시

Section 04 키보드와 마우스 이벤트 처리

- 지정된 위젯을 클릭했을 때 다른 함수 호출

Code10-14.py

```
1  from tkinter import *
2  from tkinter import messagebox
3
4  ## 함수 선언 부분 ##
5  def clickImage(event) :
6      messagebox.showinfo("마우스", "토끼에서 마우스가 클릭됨")
7
8  ## 메인 코드 부분 ##
9  window = Tk()
10 window.geometry("400x400")
11
12 photo = PhotoImage(file = "gif/rabbit.gif")
13 label1 = Label(window, image = photo)
14
15 label1.bind("<Button>", clickImage)
16
17 label1.pack(expand = 1, anchor = CENTER)
18 window.mainloop()
```

■ event 매개변수를 활용한 마우스 이벤트 처리

- 마우스를 클릭할 때마다 어떤 마우스가 클릭되었는지 보여 주고 클릭한 좌표 출력

Code10-15.py

```
1  from tkinter import *
2
3  ## 함수 선언 부분 ##
4  def clickMouse(event) :
5      txt = ""
6      if event.num == 1 :
7          txt += "마우스 왼쪽 버튼이 ("
8      elif event.num == 3 :
9          txt += "마우스 오른쪽 버튼이 ("
10
11      txt += str(event.y) + "," + str(event.x) + ")에서 클릭됨"
12      label1.configure(text = txt)
13
14  ## 메인 코드 부분 ##
15  window = Tk()
16  window.geometry("400x400")
17
```

4~12행 : 마우스 클릭할 때 실행될 이벤트 함수 선언

20행 : 마우스 클릭하면 함수 호출

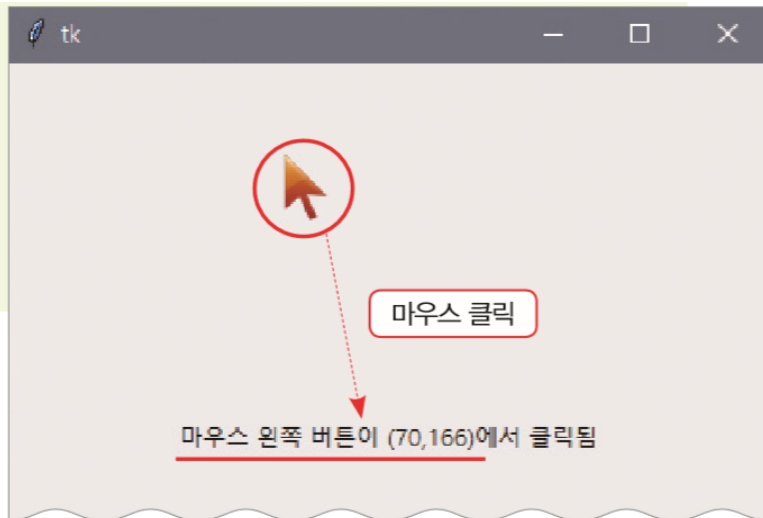
6~9행 : event.num값은 마우스 왼쪽 버튼 클릭했을 때 1값 가지고, 마우스 오른쪽 버튼 클릭했을 때 2값 가짐

11행 : event.x와 event.y는 클릭한 위치의 좌표를 가짐

12행 : 18행에서 화면에 표시한 레이블의 글자 변경

Section 04 키보드와 마우스 이벤트 처리

```
18 label1 = Label(window, text = "이곳이 바뀜")
19
20 window.bind("<Button>", clickMouse)
21
22 label1.pack(expand = 1, anchor = CENTER)
23 window.mainloop()
```



■ 키보드 이벤트 기본 처리

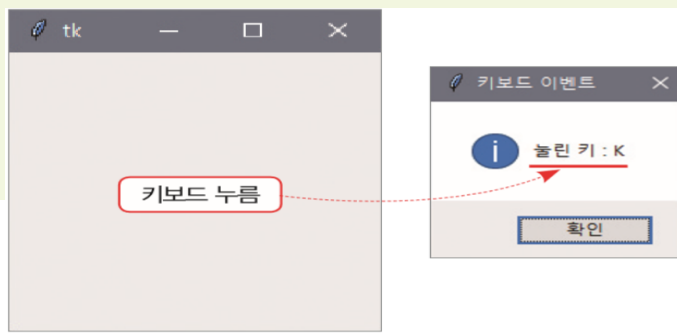
- 키보드 이벤트는 위젯에서 키보드가 눌리면 발생

Code10-16.py

```
1 from tkinter import *
2 from tkinter import messagebox
3
4 ## 함수 선언 부분 ##
5 def keyEvent(event) :
6     messagebox.showinfo("키보드 이벤트", "눌린 키 : " + chr(event.keycode))
7
8 ## 메인 코드 부분 ##
9 window = Tk()
10
11 window.bind("<Key>", keyEvent)
12
13 window.mainloop()
```

5~6행 : 키보드가 눌릴 때 작동하는 함수 선언
event.keycode에는 눌러진 키의 숫자 값 들어 있으므로
chr() 함수를 사용하면 문자로 변환

11행 : <Key> 이벤트를 윈도우창에 사용



■ 키보드 이벤트

키보드 작동	이벤트 코드
모든 키를 누를 때	<Key>
특수 키를 누를 때	<Return>, <BackSpace>, <Tab>, <Shift_L>, <Control_L>, <Alt_L>, <Pause>, <Caps_Lock>, <Escape>, <End>, <Home>, <Left>, <Right>, <Up>, <Down>, <Num_Lock>, <Delete>, <F1>~<F12> 등
일반 키를 누를 때	a~z, A~Z, 0~9, <space>, <less>
화살표 키와 조합	<Shift-Up>, <Shift-Down>, <Shift-Left>, <Shift-Right> 등

- Enter 를 처리하려면 Code10-16.py에서는 11행의 <Key> 대신 <Return>을 사용
- 대·소문자 등도 구분해서 처리 가능
- 소문자 r 은 11행의 <Key> 대신에 r을 사용해 처리
- 일반 키를 누를 때 주의할 점은 SpaceBar 는 <Space>로, < 는 <less>로 사용

■ 메뉴의 생성

■ 메뉴의 구성 개념과 형식

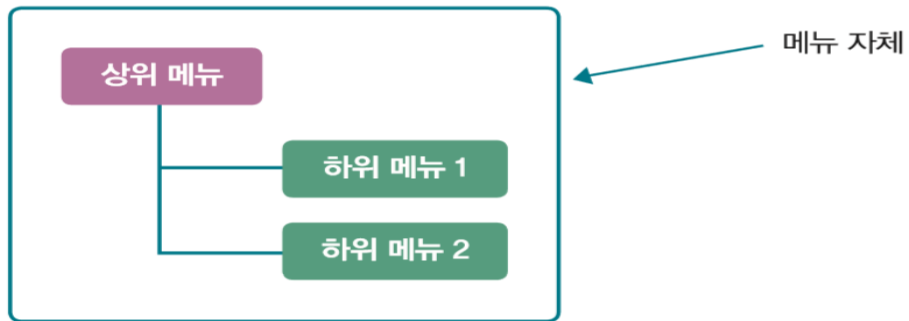


그림 10-1 메뉴의 구성 개념도

```
메뉴자체 = Menu(부모윈도)
부모윈도.config(menu = 메뉴자체)
```

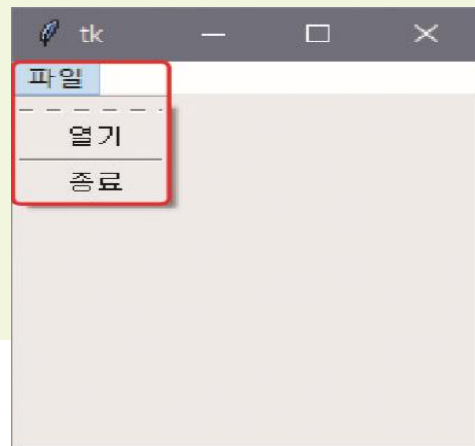
```
상위메뉴 = Menu(메뉴자체)
메뉴자체.add_cascade(label = "상위메뉴텍스트", menu = 상위메뉴)
상위메뉴.add_command(label = "하위메뉴1", command = 함수1)
상위메뉴.add_command(label = "하위메뉴2", command = 함수2)
```

Section 05 메뉴와 대화상자

- [파일] 메뉴 아래에 [열기]와 [종료] 하위 메뉴가 있는 코드

Code10-17.py

```
1 from tkinter import *
2
3 window = Tk()
4
5 mainMenu = Menu(window)
6 window.config(menu = mainMenu)
7
8 fileMenu = Menu(mainMenu)
9 mainMenu.add_cascade(label = "파일", menu = fileMenu)
10 fileMenu.add_command(label = "열기")
11 fileMenu.add_separator()
12 fileMenu.add_command(label = "종료")
13
14 window.mainloop()
```



Section 05 메뉴와 대화상자

- 메뉴를 선택 하면 작동할 수 있도록 코드 추가

Code10-18.py

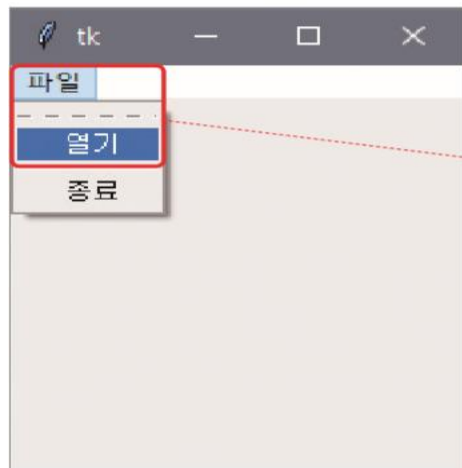
```
1 from tkinter import *
2 from tkinter import messagebox
3
4 ## 함수 선언 부분 ##
5 def func_open() :
6     messagebox.showinfo("메뉴선택", "열기 메뉴를 선택함")
7
8 def func_exit() :
9     window.quit()
10    window.destroy()
11
12 ## 메인 코드 부분 ##
13 window = Tk()
14
15 mainMenu = Menu(window)
16 window.config(menu = mainMenu)
17
```

5~6행 : [열기] 메뉴 선택하면 간단한
메시지창 열림

8~10행 : [종료] 메뉴를 선택하면 프로그램이
종료

Section 05 메뉴와 대화상자

```
18 fileMenu = Menu(mainMenu)
19 mainMenu.add_cascade(label = "파일", menu = fileMenu)
20 fileMenu.add_command(label = "열기", command = func_open)
21 fileMenu.add_separator()
22 fileMenu.add_command(label = "종료", command = func_exit)
23
24 window.mainloop()
```



■ 대화상자의 생성과 사용

- tkinter.simpledialog 모듈을 임포트한 후 askinteger() 및 askstring() 등을 사용

Code10-19.py

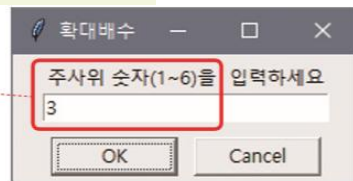
```
1 from tkinter import *
2 from tkinter.simpledialog import *
3
4 ## 함수 선언 부분 ##
5 window = Tk()
6 window.geometry("400x100")
7
8 label1 = Label(window, text = "입력된 값")
9 label1.pack()
10
11 value = askinteger("확대배수", "주사위 숫자(1~6)을 입력하세요", minvalue = 1, maxvalue = 6)
12
13 label1.configure(text = str(value))
14 window.mainloop()
```

2행 : tkinter.simpledialog 모듈 임포트

8~9행 : 레이블을 하나 준비

11행 : askinteger("제목", "내용", 옵션) 함수로 정수 입력

13행 : 입력받은 숫자를 문자열로 변경해서 레이블에 씀



■ 그림 파일인 GIF 파일을 선택하는 코드

- Code10-20.py는 Code10-19.py의 2행과 11행만 변경

Code10-20.py

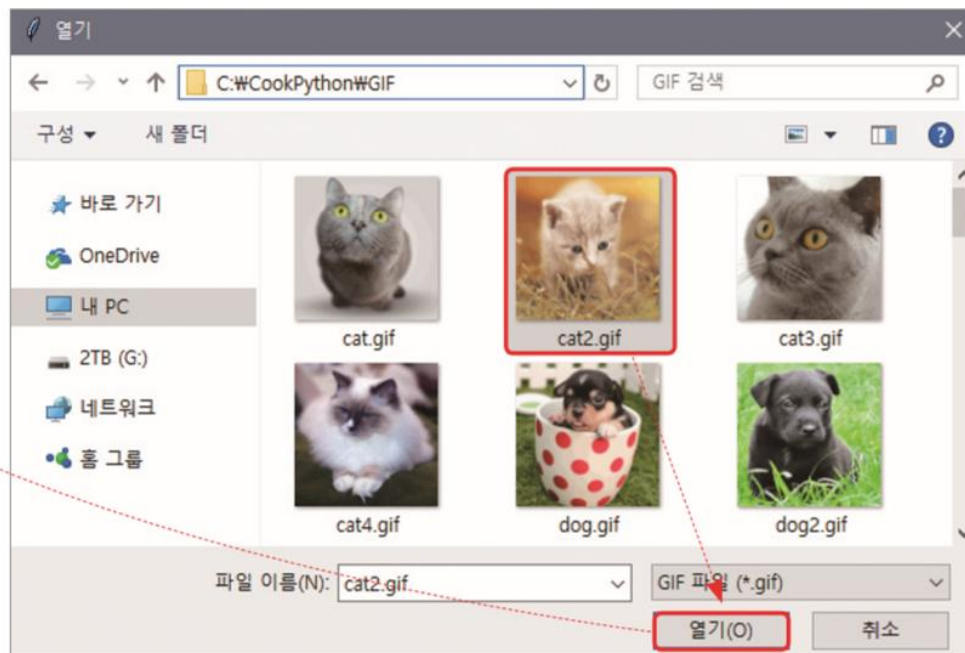
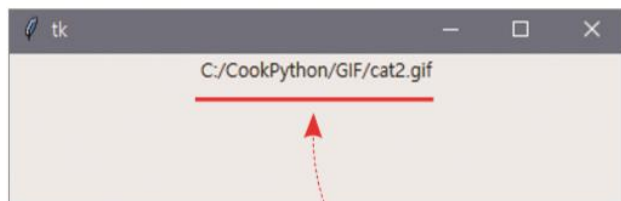
```
1  from tkinter import *
2  from tkinter.filedialog import *
3
4  ## 함수 선언 부분 ##
5  window = Tk()
6  window.geometry("400x100")
7
8  label1 = Label(window, text = "선택된 파일 이름")
9  label1.pack()
10
11 filename = askopenfilename(parent = window, filetypes = (("GIF 파일", "*.gif"),
12                                     ("모든 파일", "*.*")))
13
14
15 window.mainloop()
```

2행 : tkinter.filedialog 모듈 импорт

11행 : askopenfilename() 함수 사용

13행 : filename을 출력

Section 05 메뉴와 대화상자

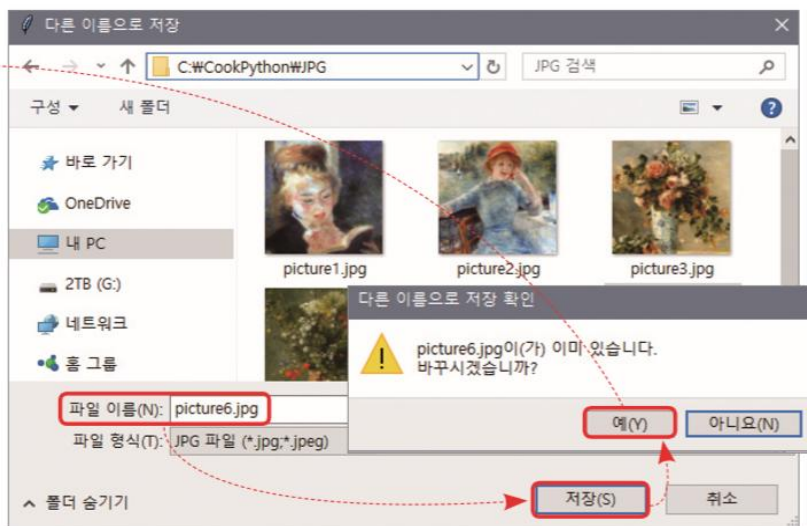
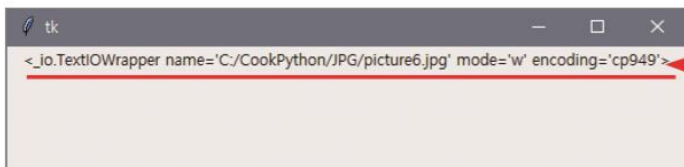


Section 05 메뉴와 대화상자

- Code10-20.py의 11~13행 변경하고 실행

Code10-21.py

```
11 saveFp = asksaveasfile(parent = window, mode = "w", defaultextension = ".jpg",  
    filetypes = (("JPG 파일", "*.jpg;*.jpeg"), ("모든 파일", "*.*")))
12 label1.configure(text = saveFp)
13 saveFp.close()
```



■ [프로그램 2]의 완성

- 메뉴 처리와 파일 처리가 핵심

Code10-22.py

```
1  from tkinter import *
2  from tkinter.filedialog import *
3
4  ## 함수 선언 부분 ##
5  def func_open() :
6      filename = askopenfilename(parent = window, filetypes = (("GIF 파일", "*.gif"),
7          ("모든 파일", "*.*")))
8      photo = PhotoImage(file = filename)
9      pLabel.configure(image = photo)
10     pLabel.image = photo
11
12 def func_exit() :
13     window.quit()
14     window.destroy()
15
16 ## 메인 코드 부분 ##
17 window = Tk()
18 window.geometry("400x400")
```

Section 05 메뉴와 대화상자

```
18 window.title("영화 감상하기")
```

```
19
```

```
20 photo = PhotoImage()
```

```
21 pLabel = Label(window, image = photo)
```

```
22 pLabel.pack(expand = 1, anchor = CENTER)
```

```
23
```

```
24 mainMenu = Menu(window)
```

```
25 window.config(menu = mainMenu)
```

```
26 fileMenu = Menu(mainMenu)
```

```
27 mainMenu.add_cascade(label = "파일", menu = fileMenu)
```

```
28 fileMenu.add_command(label = "파일 열기", command = func_open)
```

```
29 fileMenu.add_separator()
```

```
30 fileMenu.add_command(label = "프로그램 종료", command = func_exit)
```

```
31
```

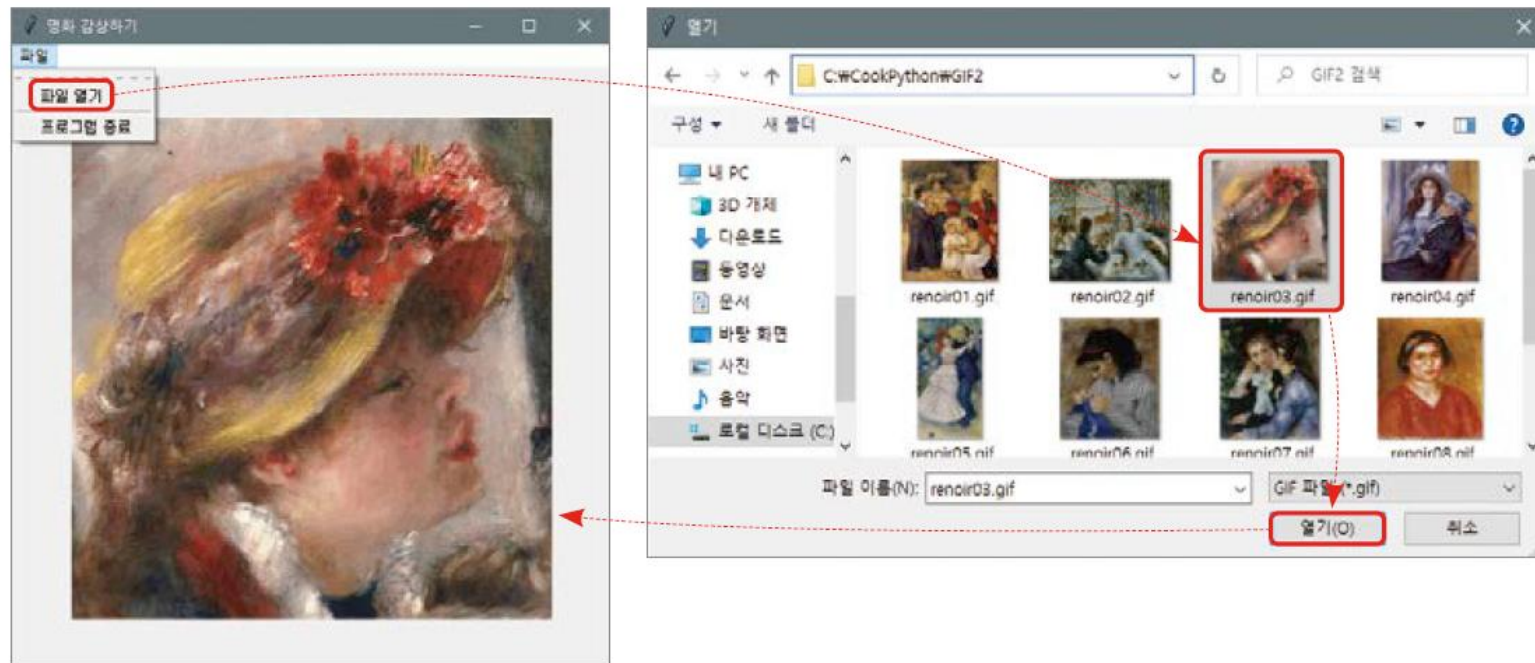
```
32 window.mainloop()
```

20행 : PhotoImage()에 별도의 매개변수 없이 빈 그림만 준비

20~22행 : 선택한 GIF 윈도우창에 출력하려고 레이블 준비

24~30행 : 준비하는 메뉴는 앞에서 실습한 내용과 동일

Section 05 메뉴와 대화상자





`print("Hello, world!")`



`<class 'bool'>`

Thank You



`if __name__ == '__main__':`
 `main()`